



Resumão de JavaScript



Status

Não iniciada



Aula Completa de JavaScript para Iniciantes

JavaScript é a linguagem que dá **vida e interatividade** às páginas web.

Vamos começar pelo básico: como incluir o JS no HTML e depois aprender os principais comandos.

Como adicionar JavaScript no HTML

Inline (na própria tag)

```
<button onclick="alert('Olá!')">Clique aqui</button>
```

Dentro do HTML

```
<script>
  alert("Olá, mundo!");
</script>
```

Arquivo separado

```
<script src="script.js"></script>
```

Tipos de dados

JavaScript trabalha com diferentes tipos de dados para representar informações:

- **String** → texto entre aspas: `"Olá"` ou `'JavaScript'`
- **Number** → números inteiros ou decimais: `42`, `3.14`
- **Boolean** → verdadeiro ou falso: `true` ou `false`
- **Array** → lista de valores: `["maçã", "banana", "uva", 1, 10, 5]`
- **Object** → conjunto de propriedades: `var pessoa = {nome: "João", array = ["maçã", "banana", 1, 5], idade: 25, nomes: {pessoa1: "Leonel", pessoa2: "Fabricio"}}`

```
typeof "texto 20 #!m³$ra↔³" // "string"
typeof 10 // "number"
typeof true // "boolean"
typeof [] // "array" "object"
typeof {} // "object"
typeof null // "object" ❌ (bug histórico do JS)
typeof undefined // "undefined"
typeof function(){} // "function"
```

1 Variáveis

Explicação:

Variáveis servem para **guardar valores**.

- `let` → valor pode mudar.
- `const` → valor fixo.
- `var` → mais antigo, menos usado hoje.

Exemplos

```
// Exemplo 1: Criando variáveis
let nome = "Ana";
const ano = 2025;
console.log(nome, ano);

// Exemplo 2: Alterando valor de uma variável
let idade = 20;
idade = 21;
console.log("Idade atual:", idade);

// Exemplo 3: Concatenando valores
let cidade = "São Paulo";
console.log("Nome: " + nome + ", Cidade: " + cidade);
```

Exercícios

1. Crie variáveis para armazenar seu nome, idade e cidade, e mostre tudo no console.
2. Use uma variável `let` para contar a quantidade de livros lidos, atualize o valor e mostre o resultado.

Operadores

Explicação:

São usados para **calcular** ou **comparar valores**.

- Aritméticos: `+` `-` `*` `/` `**` `%`
- Comparação: `==` `===` `!=` `!==` `>` `<` `>=` `<=`

Exemplos

```
// Exemplo 1: Operadores aritméticos
let a = 10, b = 3;
console.log("Soma:", a + b);
console.log("Resto:", a % b);

// Exemplo 2: Comparações
console.log(5 > 3); // true
console.log(5 == "5"); // true
console.log(5 === "5"); // false

// Exemplo 3: Operadores em variáveis
let preco = 100;
let desconto = 20;
console.log("Preço final:", preco - desconto);
```

Exercícios

1. Crie duas variáveis `x` e `y` e mostre a soma, subtração e multiplicação no console.
2. Crie um programa que verifique se a idade de uma pessoa é maior ou igual a 18 (retorne true ou false).

3 If / else / else if

Explicação:

O `if` permite **tomar decisões** no código.

- Se a condição for **verdadeira**, executa um bloco.
- Caso contrário, executa o bloco do `else`.

Exemplos

```
// Exemplo 1: Checando idade
let idade = 18;
if (idade >= 18) {
  console.log("Pode dirigir");
}
```

```
} else {  
  console.log("Não pode dirigir");  
}  
  
// Exemplo 2: Notas  
let nota = 7;  
if (nota >= 6) {  
  console.log("Aprovado!");  
} else {  
  console.log("Reprovado!");  
}  
  
// Exemplo 3: Vários if  
let hora = 15;  
if (hora < 12) {  
  console.log("Bom dia")  
  let nome = "Leonel"  
} else if (hora < 18) {  
  console.log("Boa tarde");  
} else {  
  console.log("Boa noite");  
}
```

Exercícios

1. Crie um programa que guarda um número e diga se ele é **positivo, negativo ou zero**.
2. Crie um programa que peça a nota de um aluno e diga se está **aprovado (≥6), reprovado (<4) ou de recuperação (≥4 e <6)**.

4 Loops (Laços de repetição)

Explicação:

Laços servem para **repetir um comando várias vezes** sem precisar escrever o código repetidamente.

- `for` → quando sabemos o número de repetições.
- `while` → enquanto uma condição for verdadeira.

Exemplos

```
// Exemplo 1: For simples
for (let c = 1; c <= 5; c++) {
  console.log("Número:", c);
}

// Exemplo 2: While
let contador = 1;
while (contador <= 3) {
  console.log("Contagem:", contador);
  contador++;
}

// Exemplo 3: Loop em array
let frutas = ["maçã", "banana", "uva"];
for (let i = 0; i < frutas.length; i++) {
  console.log("Fruta:", frutas[i]);
}
```

Exercícios

1. Crie um loop que mostre todos os números de **1 a 100** no console.
2. Crie um array com 5 nomes e use um loop para imprimir todos.

Funções

Explicação:

Funções agrupam comandos que podem ser **reutilizados**.

Podem receber valores (parâmetros) e devolver um resultado.

Exemplos

```
// Exemplo 1: Função simples
function saudacao() {
  console.log("Olá, mundo!");
}

saudacao();
```

```
// Exemplo 2: Função com parâmetro
function s

}
somar(5, 3);

// Exemplo 3: Função que retorna valor
function dobro(num) {
    return num * 2;
}
console.log("Dobro de 4 " é", dobro(4));
```

Exercícios

1. Crie uma função que receba um nome e mostre uma mensagem de boas-vindas.
2. Crie uma função que receba um número como parâmetro e retorne se ele é **par ou ímpar**.

Conclusão

Agora você conhece os principais blocos de construção do JavaScript:

- Variáveis
- Operadores
- If/Else if / Else
- Loops
- Funções

 Com esses fundamentos, já é possível criar programas interativos e começar a programar de verdade!